



Вахты 2026:
бухгалтерский
и налоговый учет

Артур Равкин

Налоговые, налоговые
выплаты и другие
расчеты

PDF

smartway

Учет вахт для крупных компа-
ний с несколькими юрисдикциями

smartway.today РЕКЛАМА · 16+

Найдите курс под свою цель

Войти



just_ai

8 сен в 11:44

Как мы превратили правила команды разработки в инструкции для LLM-агента

Средний

11 мин

11K

Блог компании Just AI, Программирование*, DevOps*, Искусственный интеллект, Управление разработкой*

Кейс

Привет, Хабр! На связи команда разработки Just AI. Сегодня расскажем, как мы автоматизировали часть задач в процессах разработки с помощью LLM-агента. Не генерацию функций и не написание кода по промпту — с этим кодинг-ассистенты уже неплохо справляются. Нас интересовало другое: что происходит с задачей до и после того, как разработчик закончил писать код.

Нужно создать merge request, описать изменения, запустить сборку, дождаться результата, приложить ссылку к задаче в Jira, перевести статус, передать задачу тестировщику и списать время и дождаться ревью. На небольшой задаче таких действий набирается около десяти. Нужно помнить порядок шагов, договоренности команды и постоянно переключаться между разными инструментами.

Мы решили автоматизировать этот слой с помощью LLM-агента. Для этого описали правила команды обычными текстовыми инструкциями и подключили к ним инструменты для работы с Jira, GitLab, Jenkins и Sentry.

В этой статье расскажем, как мы разобрали эти действия на слои, какие ограничения пришлось добавить и что в итоге изменилось в работе команды.

Где теряется время при закрытии задачи

Для нас рутина — повторяющееся действие, которое не двигает ни продукт, ни разработчика. Его просто нужно сделать.

По оценке команды, на эту работу уходит примерно два-три часа из восьмичасового рабочего дня. Точных замеров у нас нет, поэтому это не строгая метрика, а ориентировочная оценка. При этом сама работа устроена примерно одинаково у всех, поэтому порядок затрат времени можно использовать как ориентир.

Вот как закрывалась одна стандартная небольшая задача до появления агента:



1. Открыть задачу в Jira, прочитать описание и комментарии.
2. Сделать фикс, запустить код и назвать ветку по правилам.
3. Описать merge request для разработчиков.
4. Описать в задаче, что сделано и почему, уже для тестировщика.
5. Пойти в Jenkins, запустить сборку, дождаться результата.
6. Приложить ссылку на сборку к задаче.
7. Пинговать коллег, чтобы посмотрели MR.
8. Передать задачу тестировщику, проверить, что окружение поднято на стенде.
9. Перевести задачу в нужный статус и списать время.

Получается несколько систем и множество переключений между ними. На каждом шаге нужно помнить не только саму операцию, но и мелкие договоренности команды: какой комментарий написать, когда можно запускать сборку, что проверить перед мержем и что считать успешным результатом. Потеря времени в передаче задачи между людьми и системами.

LLM умеет работать с кодом. А с процессом?

Когда мы начали использовать LLM, многие задачи стали занимать заметно меньше времени: поиск по коду и документации, разбор чужого кода, поиск причины ошибки, реализация несложных фич. Код мы фактически перестали писать руками.

Все, что вокруг, осталось как было — и даже добавило работы:

- контекст задачи приходилось приносить модели вручную: скопировать описание из Jira, потом комментарии, потом скачать и приложить файлы;
- готовый комментарий модель написать могла, но забрать его и вставить в задачу нужно было самому.

Логичный следующий шаг — подключить MCP к трекеру, выдать токен и разрешить писать в задачу напрямую. Здесь обнаружилась другая проблема: **сам доступ к API ничего не говорит агенту о том, как принято работать в конкретной команде.**

Например, модель может написать в задаче подробный комментарий на несколько абзацев. Формально все правильно, но тестировщику он не нужен. Ему достаточно знать, что изменилось и что сборка прошла.

Главный вывод из этого этапа: **просто доступа к API недостаточно.** MCP отвечает только на вопрос «что можно сделать технически». Но этого мало, чтобы выполнить задачу так, **как принято в нашей команде:** порядок действий, правила и ограничения нужно еще где-то описать.



Архитектура системы в три слоя

Отталкиваясь от собственной рутины, мы разложили решение на три уровня.

Уровень	Где лежит	Отвечает за
Верхний — playbooks	playbooks/, _index.md	что и в каком порядке
Средний — actions	actions/	как это делать правильно
Нижний — MCP-серверы	mcp-servers/	что физически можно сделать

Нижний уровень — MCP-серверы Это вызовы API наших систем: трекера, репозитория, сборки, мониторинга. Писать их самостоятельно под каждый сервис долго, поэтому для простых интеграций мы использовали скилл-генератор из открытых наборов — в нашем случае MCP Builder. Можно словами описать, какие операции нужны: например, «сделай сервер к нашему CI — запускать сборку, получать статус и читать лог».

Над MCP находятся actions. Это атомарные действия, в которых уже описано, как пользоваться конкретным инструментом. Например, jira/comment.md отвечает не просто за отправку комментария, а за то, каким должен быть этот комментарий и что проверить после публикации.

На верхнем уровне находятся playbooks — последовательности действий для процессов, которые команда выполняет регулярно. Например: заморозить изменения, собрать проект, обновить задачу и передать ее дальше.

Отдельно задаются права агента в `.claude/settings.json`: какие операции он может выполнять самостоятельно, а перед какими должен остановиться и попросить подтверждение.

Полный путь запроса выглядит так: CLAUDE.md → индекс playbooks → playbook → action → MCP → подтверждение → действие.

Например, если разработчик пишет «собери ZB-451 и отпишись в задачу», агент сначала находит подходящий процесс, затем нужные действия и только после этого вызывает конкретные MCP-инструменты.

Такое разделение позволяет менять отдельные части независимо. Если изменился способ запуска сборки, достаточно поправить соответствующий action. Playbooks, которые используют это действие, менять не нужно.



Как устроен action

Одно логическое действие у нас — это один файл. Внутри примерно одинаковая структура: назначение, инструменты, шаги и проверка результата.

```
team-kit
├── CLAUDE.md
├── .claude
│   └── skills
│       └── mcp-builder
│           └── SKILL.md
├── .mcp.json
├── mcp-servers
│   ├── ci
│   ├── tracker
│   └── actions
│       └── ci
│           └── build.md
```

```
build.md

---
name: ci/build
destructive: true
---

## Назначение
Собрать ветку и дождаться результата.
<!-- границы: сборка — да,
      выкатка на стенд — другой файл -->

## Инструменты
- mcp_ci_get_build
- mcp_ci_trigger_build
<!-- белый список: чего нет в нем,
      агент не вызывает -->

## Шаги
1. Определи ветку. Не уверен — спроси.
2. Сборка этой ветки уже идет — не запускай вторую.
3. Запусти и дождись результата.
<!-- шаг 2 — знание про очередь,
      в API его нет -->

## Проверка
SUCCESS или FAILURE. Упала — покажи первую ошибку из лога.
<!-- иначе «вызов не упал»
      = «все получилось» -->
```

Действие целиком: назначение с границами, белый список тулов, шаги и проверка результата

Здесь есть несколько важных деталей, которые стоит учесть:

Границы пишите в назначении. «Сборка — да, выкатка на стенд — другой файл». Иначе одно действие постепенно разрастается до половины процесса, и его нельзя переиспользовать в другой цепочке.

Инструменты — это белый список. Если тул не указан в action, агент его не вызывает. Это простой способ ограничить радиус действия конкретной инструкции.

Проверка результата обязательна. Агенту недостаточно понять, что API вернул ответ без ошибки. Нужно явно описать, что считать успешным результатом.



Например: если тул вернул ID комментария, считаем, что комментарий опубликован. Если ID не вернулся, показываем ошибку и не отправляем запрос повторно.

В файлах мы также оставляем комментарии, которые объясняют, зачем появился конкретный шаг. Через месяц разработчик может увидеть непонятную строку и решить, что она лишняя. Комментарий сохраняет контекст, из-за которого этот шаг вообще появился.

Почему одно действие — один файл

Такое дробление нужно по нескольким причинам. Во-первых, экономится контекст. Под конкретное действие агенту не нужно загружать всю инструкцию по работе с трекером — достаточно одного файла.

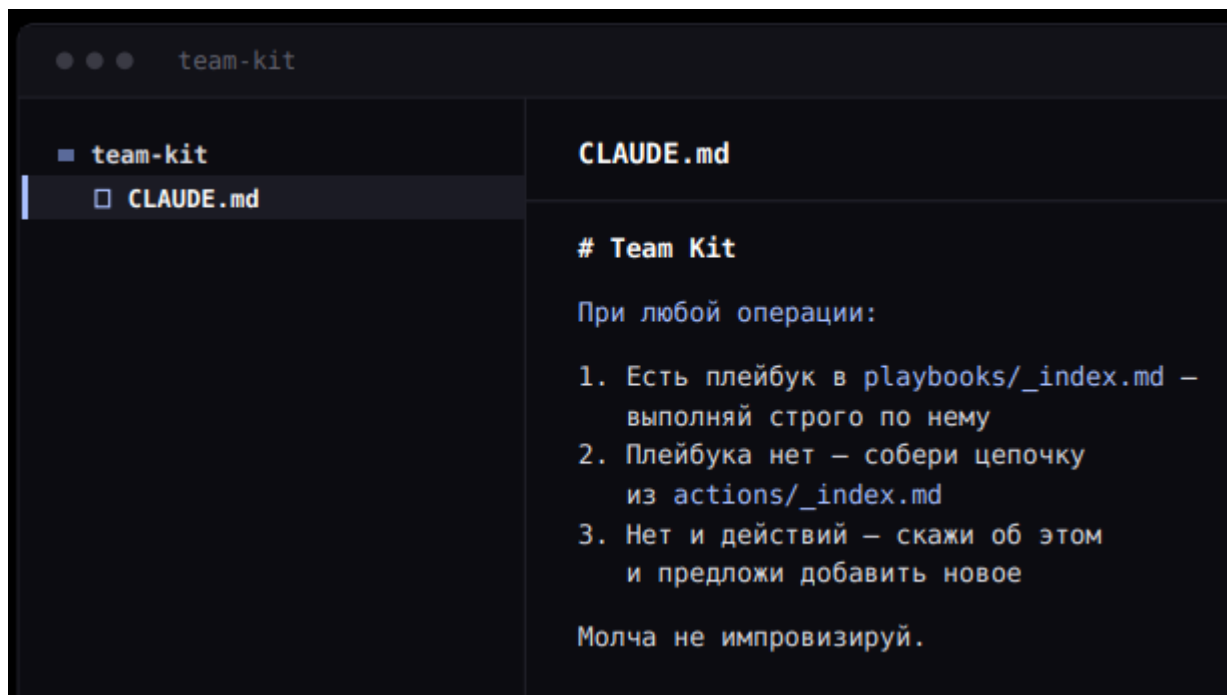
Во-вторых, чем короче инструкция, тем точнее агент ей следует. Поэтому мы уместаем описание одного действия примерно в 50-70 строчек: только конкретное действие и условия его выполнения, без лишнего контекста.

В-третьих, из атомарных действий можно собирать новые процессы. Например, запрос «сделай сборку, если все хорошо — напиши Анне в Mattermost и переведи задачу» не обязательно заранее описывать отдельным playbook. Если есть три соответствующих actions, агент может собрать цепочку сам.

Индексы инструкций и экономия контекста

Когда инструкций становится много, встает вопрос, как найти нужную и не загрузить агенту весь репозиторий. Мы сделали двухуровневый индекс.

Точка входа одна — CLAUDE.md, единственный файл, который агент читает сам, без просьбы:



Дальше запрос «напиши в задачу, что баг пофикшен» проходит так: `playbooks/_index.md` → `actions/_index.md` → `actions/tracker/comment.md` → MCP-сервер.

В индексе хранится по одной строке на действие. Агент сначала смотрит короткое описание, а целиком загружает только тот файл, который подходит для текущего запроса.

Индексы мы генерируем из шапок файлов, а не редактируем вручную. Если подходящего `playbook` нет, агент ищет действие напрямую. Если не находит и его — останавливается и спрашивает человека, а не пытается придумать новый способ работы.

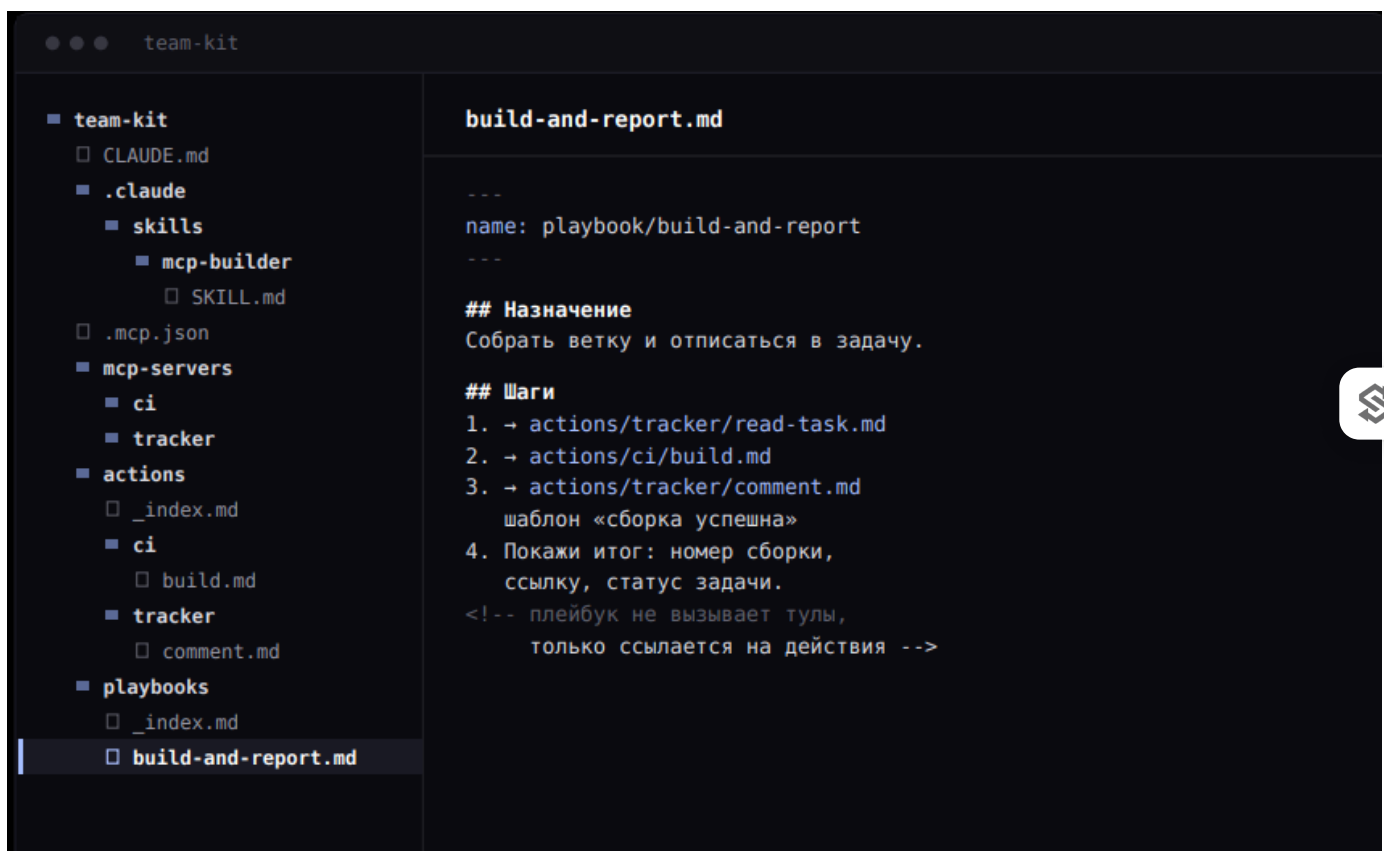
Так в контексте остаются только инструкции, которые нужны для конкретного запроса.

Зачем нужен отдельный слой `Playbook`

Action помогает понять, как правильно выполнить один шаг, а `Playbook` — какие шаги и в каком порядке нужно выполнить для конкретного процесса.

Например, после успешного ревью команда всегда проходит одну и ту же цепочку: проверить MR, заморозить изменения, запустить сборку, дождаться результата, оставить комментарий в задаче и изменить ее статус.

`Playbook` описывает этот порядок, но не содержит вызовов API и подробной логики отдельных действий. Он только ссылается на `actions`:



Playbook не вызывает тулы, он только ссылается на действия. Правка в build.md починит все цепочки, где участвует сборка

Это дает несколько преимуществ:

- **Изменения в одном месте распространяются на все процессы.** Если мы поменяли правила сборки в actions/ci/build.md, все playbooks, которые используют это действие, получают новую логику.
- **Playbook не превращается в жесткий скрипт.** Если разработчик попросил не выполнять один из шагов, агент может пропустить его, сохранив остальные проверки и порядок.
- **Playbook не обязателен.** Если готового процесса нет, агент может собрать цепочку из отдельных actions — тогда порядок придумывает он, а знание о том, как правильно, все равно берет из файлов действий. Playbook нужен там, где порядок шагов важен и должен быть одинаковым у всей команды.

Как агент закрывает задачу

Возьмем простую задачу из ежедневной работы. Разработчик пишет: «Замержи ZB-12345».

Для этого процесса у нас есть playbook. Агент проходит несколько шагов:

1. Сам находит merge request, связанный с задачей.
2. Проверяет апрувы (у нас в компании нужно два обязательных апрува).

3. Смотрит, что все четыре обсуждения закрыты.
4. Проверяет конфликты.
5. Если все в порядке, он доходит до первого места, где требуется участие человека, — показывает, что собирается сделать мерж в main, и ждет подтверждения.
6. После подтверждения мержит в main.
7. Запускает сборку и дожидается результата.
8. Когда сборка готова, он формирует комментарий для задачи и снова останавливается. В комментарии будет короткая сводка: задача с мержена в main, статус и ссылка на сборку, компонент и ветка. Такой формат мы заранее описали в `actions/tracker/comment.md`, поэтому агент не пытается каждый раз сочинять комментарий заново.
9. Показывает превью и ждет подтверждения.
10. После второго подтверждения он публикует комментарий, переводит задачу в статус «интеграция» и списывает время.



Формат комментария заранее описан в `actions/tracker/comment.md`, поэтому агент не сочиняет его заново для каждой задачи. В нем будет короткая сводка: задача с мержена в main, статус и ссылка на сборку, компонент и ветка.

Получается, что человек нужен только в двух местах — там, где агент меняет состояние во внешних системах и действие уже нельзя незаметно отменить. Все остальное, включая чтение задачи и ожидание сборки, проходит автоматически.

Раньше эта цепочка занимала 15–20 минут, а с инструкциями — около пяти.

При этом разработчик может изменить отдельный шаг прямо в запросе. Например: «Замержи ZB-12345, но не списывай время»

Агент пройдет ту же цепочку, но не будет выполнять последнее действие. При этом остальные проверки и порядок шагов сохранятся: они описаны в отдельных `actions` и не зависят от формулировки запроса.

Как мы автоматизировали дежурство

Еще один сценарий — разбор ошибок из Sentry (сервиса мониторинга ошибок приложений). Здесь мы сделали отдельный изолированный сценарий со своей директорией и правами. Агент не должен выходить за его пределы.

По команде «Собери ошибки Sentry» он:

1. Собирает ошибки за последние 14 дней.
2. Отсеивает внешние ошибки, которые не связаны с нашим кодом, например ошибки расширений браузера.
3. Объединяет дубли — одинаковые ошибки от разных пользователей.
4. Распределяет ошибки по экспертизе. У разработчиков разные зоны ответственности, поэтому ошибку в биллинге нет смысла отдавать человеку, который занимается редактором.
5. Создает задачи на четырёх разработчиков и указывает спринт.
6. Создает merge request с обновлением конфигурации Sentry, чтобы исключенный шум больше не приходил.
7. Отправляет отчет в чат дежурства.



Раньше дежурство занимало **50–60 минут**: проанализировать каждую ошибку, разложить по людям, выполнить операции в интерфейсе, обновить конфигурацию, оформить задачи. Сейчас все удастся сделать за **10–15 минут**, из которых человеческая часть — посмотреть merge request с исключениями и прочитать отчет. Создание четырех задач, на которое уходило около десяти минут, выполняется примерно за полминуты.

Как ограничили права агента

Автоматизация не означает, что агенту нужно разрешить все. **Мы разделили ограничения на два уровня:**

Мягкие ограничения задаются прямо в action. Например, инструкция может требовать показать превью комментария и дождаться ответа перед публикацией. Это правило держится на том, что агент читает и соблюдает инструкцию.

Жесткие ограничения задаются в `.claude/settings.json` и не зависят от текста action.

```
{
  "permissions": {
    "allow": ["mcp_ci_get_build", "mcp_tracker_get_issue"],
    "ask":   ["mcp_ci_trigger_build", "mcp_tracker_add_comment"]
  }
}
```

Объяснить с  SourceCraft

Получать данные агент может самостоятельно, а перед изменением чего-либо во внешней системе должен запросить подтверждение. Поэтому статус сборки он проверяет

сам, а новую сборку запускает только после согласия разработчика.

*Про персональные данные — это отдельный вопрос. В описанном сценарии такой задачи нет: через агента не проходят ПД. Если делать такую систему для процессов с ПД, понадобится отдельный слой, который будет проверять данные до отправки запроса в агентную среду.

Что сломалось и как это исправили



На наш взгляд, это самая интересная часть: на наших ошибках вы сможете заранее учесть риски. Первая версия системы работала, но мы быстро нашли несколько проблем.

Агент начал обходить инструкции. При трех слоях он игнорировал playbooks и actions и шел сразу в MCP-сервер. Так быстрее, но смысл архитектуры пропадал: знание о том, как правильно работать, находилось в файлах, которые агент просто пропускал.

Для того, чтобы решить это, мы добавили security hooks: вызвать MCP нельзя, пока в этом же запросе не прочитан файл действия.

Расширение на команды принесло новые merge-requests. Как только инструмент вышел за пределы одной команды, начали появляться свои инструкции, playbooks и actions. Общие правила смешались с командными.

Мы разделили инструкции на три уровня:

- common — общие правила компании;
- team — правила конкретной команды;
- local — личные настройки разработчика.

Например, время у нас списывается по общему правилу, поэтому это common. Формат merge request может отличаться у команд — это team. А личные предпочтения разработчика можно вынести в local.

По нашей оценке, около 90% инструкций общие, остальное переопределяется на уровне команды или разработчика.

Это важно учитывать при масштабировании системы: правило одной команды не стоит делать корпоративным стандартом. Общий стандарт, который не совпадает с реальностью команды, начнет мешать и вызывает поток правок в стиле «мы тут поправили под себя».

Скрипты ломали архитектуру. Разработчики добавляли много собственных скриптов, и система становилась нестабильной. Пришлось закрыть это тестами: pre-commit и pre-push, которые проверяют, что архитектура цела, и прогоняют скрипты.

Инструкция описывает не все, что человек имеет в виду. Человек выполняет процесс и держит часть правил в голове. Когда эти правила приходится записывать для агента, выясняется, что половина из них никогда не была сформулирована.

Поэтому инструкции приходится дописывать после реальных ошибок агента.



Что изменилось после автоматизации

Полноценных замеров мы не вели: таймеров на переключения не вешали и клики не считали. Но собрали наблюдения по конкретным процессам:

Процесс	Было	Стало
Закрытие задачи (цепочка в трех системах)	около десяти переключений, 15–20 минут	одна фраза, около пяти минут, человек подтверждает два шага
Дежурство по ошибкам	50–60 минут	10–15 минут
Вопрос из чата про баг	руками донести контекст: задача, ветка, стенд, логи	ссылка на обсуждение, без копипаста
Онбординг	мерж — на третий день	мерж по регламенту в первый день
Рутина в день	2–3 часа	около часа и меньше

Раньше, чтобы разобрать проблему из корпоративного мессенджера, нужно было скопировать тред, скопировать задачу из трекера, принести все в модель и объяснить, что с этим делать. Сейчас достаточно сказать, в какой группе идет обсуждение и какая задача с ним связана, — остальное агент соберет сам.

С онбордингом тоже получился классный эффект: новый сотрудник может не искать человека, который объяснит процесс мержа, а попросить агента выполнить его по регламенту и посмотреть, какие шаги тот проходит.

Что нужно, чтобы собрать такой слой у себя

Для такой системы нужны три компонента:

1. Любая агентная среда, в которой вам комфортно работать.

Слой инструкций не привязан к конкретной модели: агентная среда может быть любой — Claude Code, Codex, OpenCode. Но модели нужно откуда-то вызывать, и на проектах вроде этого удобно иметь одну точку доступа.

Мы используем наш продукт **Caila**, платформу Just AI, через которую идут обращения к 350+ моделям. Для описанного здесь сценария это удобно по трем причинам: один `base_url` и один ключ на все модели, возможность сменить модель под задачу без переписывания обвязки, и видимые расходы в разбивке по моделям и запросам.



1. **МСП-серверы** к вашим системам. Не обязательно разрабатывать их с нуля: можно взять готовый скилл-генератор и описать словами, какие операции нужны.

2. **Инструкции** — обычный текст в репозитории.

Сами инструкции тоже можно частично генерировать. Мы используем отдельный скилл, который знает нашу архитектуру и раскладывает описанный процесс по слоям: что должно стать МСП-инструментом, что — action, а что — playbook. Разработчик описывает процесс словами, смотрит получившийся план и согласовывает его.

Распространение внутри компании у нас устроено так: клонировать репозиторий и запустить setup-скрипт — он подключает МСП-серверы и просит выпустить и вставить токены.

Сейчас четыре команды разработки используют эту систему, добавляют свои инструкции и сценарии дежурств. Тестировщики тоже перешли на нее и перенесли инструкции из отдельного инструмента.

Если захотите повторить такой подход у себя, мы собрали **отдельный гайд**. В нем восемь файлов на примере команды разработки, по одному на каждый слой архитектуры, с готовыми шаблонами действия, playbooks и настроек прав.

Если вы собирали похожий слой у себя, интересно сравнить решения — делитесь в комментариях.

А если хочется посмотреть, что еще можно делать с LLM в разработке, заходите в наш **чат для разработчиков**. Там анонсируем эфиры и вебинары, рассказываем о новых возможностях продуктов и делимся практическими кейсами.

Теги: `claude code`, `mcp-server`, `llm`, `ai-агенты`, автоматизация разработки, `jira`, `gitlab`, `skills`, `sentry`, `model context protocol`

Редакторский дайджест

Присылаем лучшие статьи раз в месяц



Электронпочта

Подписаться

Оставляя почту, я принимаю Политику конфиденциальности и даю согласие на получение рассылок



16K+

Охват за 30 дней

Just AI

Сайт



16K+

1

Охват за 30 дней

Карма

Компания Just AI @just_ai

Пользователь

Подписаться



Telegram

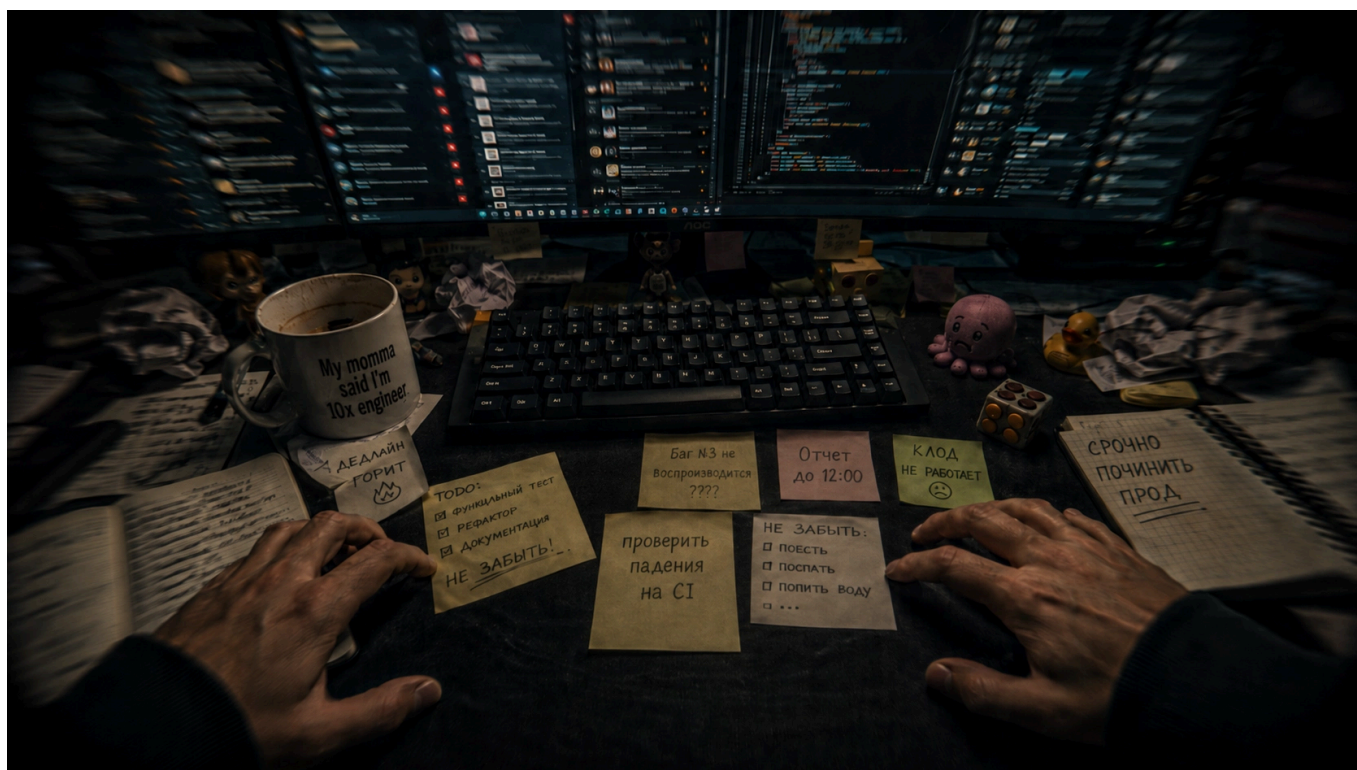


Комментарии 9

ЕЖЕДНЕВНЫЙ ХАБР | 11 СЕНТ 2026



55K



Я попал к психиатру из-за кодинга с AI

Со стороны программирование звучит как медитативное занятие. Сидишь себе за компом целый день, слушаешь музыку, пьешь вкусный кофе. У все меня примерно так и происходит. Сначала я не спеша пытаюсь выстроить в голове то, что требуется сделать. Именно «не спеша». И не потому...

Как я сделал ремастер «Том и Джерри» в 4K за два года: дубль два

В Red Hat высказали опасения по поводу сохранения ментальных способностей вайбкодеров

Отвратительные вопросы для Гофера

Ещё

3

Публикации



ЛУЧШИЕ ЗА СУТКИ ПОХОЖИЕ

 **kee_real**
22 часа назад

Я попал к психиатру из-за кодинга с AI

 **Простой**  8 мин  69K

Мнение

 **+112**  52  151

 **OzmaLee**
19 часов назад

Атомград на болотах: планировка и архитектура Припяти

 **Простой**  14 мин  14K

Ретроспектива

 **+55**  17  15

 **chrevato**
5 часов назад

Как устроен IPMI-мониторинг серверов: сбор метрик и интеграция с Grafana

 **Средний**  12 мин  4.4K

Тutorial

 **+31**  8  0

 **inetstar**
6 часов назад

Работа с гигантской памятью из Go на выделенном сервере

 Средний  29 мин  5.6K

Тutorial

 +31

 7

 1



AmneziaLover
20 часов назад

Жизнь после смерти или «Как я провёл лето»

 Простой  14 мин  13K

 +26

 27

 8



ladapriora
21 час назад

Whisper больше не нужен, русская диктовка мгновенно и без видеокарты

 Простой  3 мин  10K

Кейс

Из песочницы

 +26

 86

 29



pet67
7 часов назад

Открываем претрейн Alice AI Search: как устроена модель быстрых ответов Алисы на Поиске

 14 мин  7.3K

 +25

 6

 4



orchidfiles
17 часов назад

Альтернативы Хабра в англоязычном интернете

 Простой  7 мин  11K

Обзор

 +25

 42

 23





cnet

6 часов назад

Я радиоволна, радиоволна... Или о «блинкерах» для сотовых телефонов — продолжателях традиций детекторных приёмников

🕒 7 мин

👁 5.3K

📌 +22

🔖 8

💬 3



MaFrance351

5 часов назад

«Наберите код»: история и устройство самого массового кодового замка СССР

🔗 Средний

🕒 4 мин

👁 4.2K

Обзор

📌 +16

🔖 4

💬 6

Базовый сетевой минимум и роскошный облачный максимум: разбираем структуру VPC

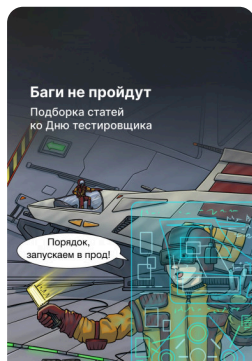
Турбо

Показать еще

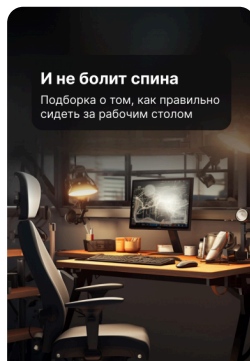
ИСТОРИИ



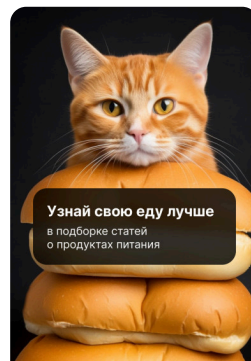
Проблемы?
Внимательно вас слушаю



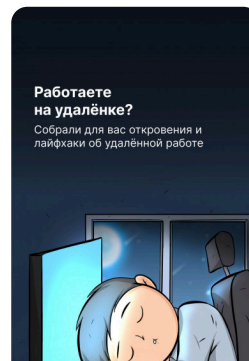
Жизнь без багов



Ваша спина очень важна для вас



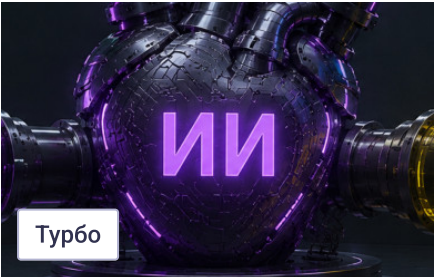
Узнай свою еду лучше



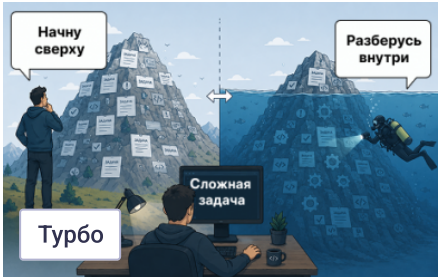
Мифы и легенды удалёнки



«Зе под



Почему удачный ИИ-кейс не масштабируется



Аквалангист vs скалолаз? Тест на ИТ-архетип



Краш-тест балансировщика: T-Rex и UDP



 [Настройка языка](#)

[Техническая поддержка](#)